



PROYECTO #1: Vista de Cazador.

Informe Técnico de Implementación.

Solución presentada y desarrollada por Rafael Paul Rodríguez Romero,
de cédula V30049983 y perteneciente a la Sección C2.

INFORME TÉCNICO DE PROYECTO:

*Inicios de Verano, año 350 DF
División de Algoritmia de Combate
Ciudadela de Havlev
Alto Mando de Arcanobastrum*

Dicen que la primavera en Arcanobastrum es hermosa; aquí en Havlev solo tenemos barro y código. El Equipo Delta ha finalizado el módulo "Vista de Cazador". No ha sido difícil superar las expectativas; digamos que la barra no estaba muy alta. Hemos sacado la basura inservible, optimizado los disparos y eliminado cualquier redundancia burocrática que pudiera ralentizar el procesador. Básicamente, hemos hecho que la máquina piense como nosotros. Rápido y sin dudas.

Ahora, una nota personal para el Alto Mando: no toquen mi código. Funciona con una precisión que salva vidas. Lo he probado yo mismo simulando los peores escenarios posibles, porque no pienso permitir que un error de cálculo cueste sangre en mi guardia. Créanme, este muro no va a caer. El sistema está listo para la guerra. Esperamos que ustedes también lo estén.

"Si la máquina falla, es culpa del operador. Si la máquina no falla, fui yo"

*Zaref Di Rogueri
Bellifex Tacticus - Frente Norte*

INTRODUCCIÓN

El presente proyecto **Vista de Cazador** simula un sistema de visión artificial diseñado para identificar objetivos en mapas de calor (PGM) y decidir estratégicamente cuáles atacar para maximizar el impacto bajo un límite de disparos **T**. El principal desafío técnico residió en la **naturaleza combinatoria** de la selección: con hasta 32 entidades, el crecimiento exponencial del espacio de búsqueda hacía inviable un **Backtracking** simple.

Para resolver esto, se diseñó una solución que integra **memoria dinámica** (para cualquier resolución) y un algoritmo de Backtracking optimizado con **validaciones** para mitigar la explosión combinatoria. Esta estrategia permite al programa "intuir" matemáticamente cuándo detener procesamientos innecesarios. La solución se estructura en dos fases: **detección** y **optimización** (una variante del *Knapsack Problem*). Gracias a la integración de "preguntas lógicas" y ordenamiento previo, el sistema logra descartar ramas inútiles y encontrar la solución óptima en milisegundos, garantizando eficiencia y robustez.

ASUNCIONES Y DEFINICIÓN DEL ENTORNO

Se asumió que los archivos de entrada no tendrían comentarios en el formato PGM (por el Enunciado, sección final de “ENTRADA”), que cumplirían con el estándar PGM en el formato ASCII o P2 por las dudas aclaradas en la primera sesión de preguntas, sin embargo, se decidió implementar una validación debido a que textualmente no se encontró respaldo de esta afirmación. Además, el procesamiento de la entrada tiene la capacidad de soportar profundidades de color de hasta 16 bits (65536 ‘colores de píxeles’) debido a que el formato PGM es lo máximo que soporta ([PGM](#), norma #7, sección “THE FORMAT”) para garantizar compatibilidad con casos de prueba extremos aunque, igualmente, se declaró en una sesión de preguntas que el máximo sería de 8 bits pero tampoco hay respaldo textual de ello; otro aspecto a tomar en cuenta es que se tomó como referencia que habrá como máximo un caso de 256x256 (Enunciado del Proyecto, sección Consideraciones).

Para el cálculo de propiedades geométricas, se decidió asumir que los objetivos o círculos no se solapan, no habrán dos con el mismo color, el máximo número posible será de 32, el centro de cada círculo siempre estará en una sola celda, y que serán enumerados desde el mayor hasta el menor nivel Oscuridad empezando en 1 y llegando hasta los **M** círculos recibidos respectivamente (asunciones tomada del Enunciado del Proyecto, sección Consideraciones); se asumió de igual manera que tienen al menos un píxel de distancia entre cada círculo y que el centro de la circunferencia de cada círculo a imprimir sin relleno será del color máximo definido previamente por el archivo PGM (FAQ, sesión 2, 24/11/25); además, que el valor de «**Pi Normalizado**» es dinámico, dependiente del promedio de las entidades detectadas en la escena y, por tanto, el «**Área Normalizada**» también lo es porque depende de *pi normalizado* (Enunciado, sección “ACERCA DE PI”).

Se estableció que la prioridad de selección en caso de empate técnico (misma Área Normalizada Discreta total) obedece al criterio de menor consumo de munición y mayor nivel de oscuridad (FAQ, sesión 1, 17/11/25).

Metodología Técnica

1. Uso del Formato PGM:

Para ingestar la información táctica, se diseñó el método Core de la clase **Imagen** el cual procedió a abrir el flujo de datos en modo lectura. Tras validar el *lenguaje* P2 del archivo, se decidió asignar memoria dinámica para una matriz de dimensiones **NxM** y se procedió a leer el ancho y largo respectivamente debido a que en los archivos PGM, a diferencia de lo usado comúnmente en computación «[i][j]», es con este orden el mismo usado al momento de trabajar con coordenadas cartesianas. Se optó de manera consecutiva por leer, secuencialmente, el color máximo de este archivo y los valores de intensidad de gris, almacenándolos en una estructura de datos que permitió el acceso aleatorio mediante coordenadas cartesianas, requisito indispensable para el análisis geométrico posterior.



2. Algoritmos de Detección, Análisis y Normalización:

2.1. Para detectar los círculos en la imagen, se implementó un recorrido completo de la matriz de píxeles ($N \times N$) mediante ciclos repetitivos. En lugar de usar funciones recursivas que podrían saturar la memoria en imágenes grandes, se optó por una **validación lógica de coordenadas**.

```
153 void detectarCirculos(int M)
```

El algoritmo utiliza un arreglo de visita para saber si un color ya fue procesado. Al encontrar un píxel de un círculo, se compara su posición (filas y columnas) con los valores iniciales y finales guardados anteriormente. Mediante condicionales simples, se actualizan los límites: si la coordenada actual es menor que el inicio registrado, se guarda como el nuevo inicio; si es mayor que el final, se actualiza el final. De esta forma, se obtienen las dimensiones exactas del círculo sin necesidad de almacenar todos sus puntos, asegurando que se detecte correctamente incluso si está pegado a los bordes de la imagen.

2.1.1. Se implementó el método auxiliar en la clase **Imagen** en la que valida que el píxel donde se encuentra en efecto sea un borde del mismo círculo. Esto se logró diseñar con un mecanismo sencillo de dos verificaciones en el que alguna de las coordenadas "x" y "y" actuales, representadas por "j" e "i" respectivamente, sea igual a el *inicio* (igual a 0) o el *final* (igual a [alto/ancho - 1]) del mapa. Si es afirmativa la respuesta, entonces se devuelve un true. De ser lo contrario se devuelve un false.

```
372 bool verificarBorde(int **mapa, int i, int j)
```

2.1.2. Se implementó el método auxiliar en la clase **Imagen** para garantizar los intervalos y poder medir tanto el diámetro como las posteriores características del círculo (pi, radio y centro) la cual se usó para verificar que las coordenadas iniciales y finales de cada círculo se coloquen de la manera más óptima posible, por ello se usó cada vez que se verifica que era un borde. La lógica implementada se basó en verificar:

```
345 void definirIntervalos(Circulo &circulo, int i, int j)
```

Para las ordenadas (eje Y) y abscisas (eje X), se aplicó una lógica de máximos y mínimos: se comparó la coordenada actual con los límites registrados, actualizando el inicio si el valor actual era menor, y el final si era mayor. Esto garantizó la creación de una especie de 'Bounding Box' precisa incluso para figuras irregulares que no se consideren círculos.

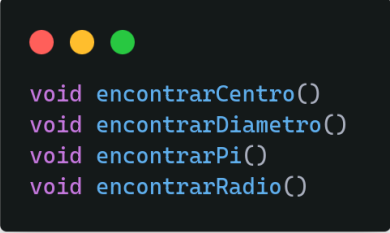
2.2. Se implementó el método Core en la clase **Imagen** cuya finalidad fue encontrar el píxel del centro de cada círculo, dibujar ese píxel con el color máximo del PGM en la circunferencia ya antes creada por *detectarCirculos()* (ver 2.1), encontrar el

```
191 void analizarCirculos(int &M)
```

diámetro, el radio y el Pi mediante funciones método auxiliares. En esta función también se decidió incluir una funcionalidad de verificación para saber si un círculo no ha sido definido ya sea por un **M** mayor a la cantidad de círculos existentes, y entonces poder actualizar el **M** para evitar tener un *mal* comportamiento en el resultado posterior.

Además, un objetivo principal de esta función también fue ordenar de mayor oscuridad a mayor claridad los círculos existentes. Se colocó después de definir cada valor característico de cada círculo y de comprobar si todos los círculos existen realmente.

2.2.1. Se implementó una serie de funciones auxiliares en la clase **Circulo** para hacer los cálculos matemáticos de la función *analizarCirculos()* (ver 2.2.).



```
void encontrarCentro()  
void encontrarDiametro()  
void encontrarPi()  
void encontrarRadio()
```

En la función *encontrarCentro()*, línea 35, se implementó la lógica simple de asumir que el **centro** o, más bien, la mitad de las abscisas, es el resultado de sumar la abscisa inicial con la abscisa final del mismo círculo y dividirla en dos y guardar el resultado en un entero. Se hizo esta misma lógica con las ordenadas para finalmente decidir en ambos casos si:

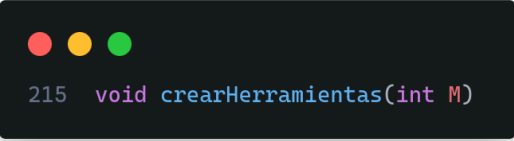
- La suma y división es mayor a 0, entonces definir el centro como ese resultado.
- Pero si es menor o igual a 0, entonces definir el centro como 0, para evitar acceder a una parte de la memoria que es ilegal.

Para la función *encontrarDiametro()*, línea 42, la lógica se basó principalmente en el ejemplo expuesto en el Enunciado inicial del Proyecto: el **diámetro** de un círculo es igual al valor absoluto de la resta entre los intervalos de las abscisas/de las ordenadas, más uno.

Para la función *encontrarPi()*, la lógica se basó principalmente en la fórmula expresada en el Enunciado del Proyecto: **Pi** es el resultado de dividir la circunferencia (forzándolo a ser float) de un círculo entre el diámetro del mismo.

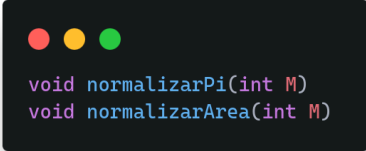
Y finalmente la lógica de implementación de la función *encontrarRadio()*, es la fórmula deducible del ejemplo expuesto en el Enunciado la cual es: el **radio** es la división entre el diámetro del círculo entre dos, más uno.

2.3. Se decidió implementar un método como función Core de la clase **Imagen** el cual se dedica a **normalizar Pi** y a normalizar el **Área** para llevarla a ser discreta.



```
215 void crearHerramientas(int M)
```

2.3.1. Se decidió diseñar dos funciones método auxiliar para **normalizar Pi** y para el **Área**.



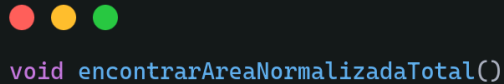
```
void normalizarPi(int M)  
void normalizarArea(int M)
```

Para encontrar el **Pi Normalizado**, fue necesario sumar los Pi_M para los M círculos existentes, y luego dividirse entre M (forzando este a ser float) para promediarse. Esta fórmula fue presentada en el Enunciado.

Para calcular el **Área Normalizada** y posteriormente el **Área Normalizada Discreta** de cada círculo, se usó la fórmula suministrada por el Enunciado, la cual fue: el área normalizada es la multiplicación de **Pi Normalizado** por el **Radio** al cuadrado. Para hacer esta **área discreta** según el Enunciado, se tuvo que aproximar siempre hacia arriba dado

que el resultado es un flotante. Para simplificar esta parte, se empleó la función `ceil()` de la librería `<cmath>`, incluida en las librerías permitidas en el Enunciado.

2.4. Para seleccionar la mejor combinación de círculos, se desarrolló una función recursiva de **Backtracking** que busca maximizar el área total respetando el límite de disparos (**T**). Para evitar que el programa realice cálculos innecesarios y tarde demasiado, se integraron



```
void encontrarAreaNormalizadaTotal()
```

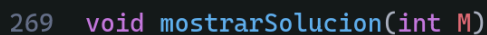
varias validaciones aritméticas antes de realizar las llamadas recursivas:

Antes de avanzar, se suma el área acumulada actual con todas las áreas de los círculos que faltan por revisar. Si el resultado de esta suma no supera al mejor área encontrada hasta el momento, el algoritmo detiene esa búsqueda inmediatamente, ya que matemáticamente no es posible encontrar una mejor solución en ese camino.

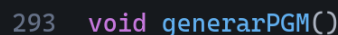
Si se alcanza el área objetivo exacto (**T**) y la cantidad de círculos usados hasta el momento no es menor a la mejor cantidad registrada, también se detiene el proceso.

Adicionalmente, como los círculos se ordenaron previamente por su nivel de **oscuridad** (de mayor a menor), el algoritmo garantiza que la primera solución válida que encuentre será la que tenga los círculos más oscuros, cumpliendo con las reglas de prioridad sin necesidad de verificaciones adicionales.

2.5. Se decidió implementar dos funciones para mostrar la solución. Una, para el resultado en **consola** (línea 269) y otra, para el resultado en **archivo PGM** (línea 293).



```
269 void mostrarSolucion(int M)
```



```
293 void generarPGM()
```

El enunciado solicitó mostrar la lista de las **áreas normalizadas discretas** de los **M** círculos seguidos de una “u” representando las unidades, se logró mediante un `cout` seguido de un `endl` en un bucle. Además, se necesitó otro bucle para imprimir la lista de **M** círculos identados con “Círculos” separados por una coma, que fueron elegidos por el backtracking.

Para lograr esto, se usó una verificación del arreglo booleano en el que sí era distinto de cero, no imprimía; para la solución de las comas después de cada círculo, sin que salga una coma al final del último círculo o al final del primer círculo en el caso de que hubiera sólo uno: se realizó la clásica variable utilizada como *bandera booleana* llamada **primer Círculo** inicializada fuera del bucle como `true`, y se procedió a verificar si “el primer círculo es falso” se imprimía la coma, y fuera de esa validación, entonces se imprimió el círculo, y su id. Por lo tanto, en el primer círculo no se imprimió, y en el último tampoco porque la validación del mismo bucle lo paró antes de llegar a esa validación de la coma.

Análisis de Resultados

El correcto funcionamiento del módulo Vista de Cazador fue verificado mediante la ejecución de una batería de nueve (9) casos de prueba diseñados específicamente para evaluar distintas dimensiones del sistema: precisión geométrica, manejo de bordes, priorización de objetivos y eficiencia del algoritmo de optimización bajo carga.

Los escenarios seleccionados cubren desde validaciones unitarias simples hasta casos de alta complejidad combinatoria, garantizando que el sistema responde correctamente ante:

- **Anomalías Geométricas:** Círculos de radio mínimo (1 píxel) y figuras ubicadas en los límites extremos de la matriz $N \times N$.
- **Restricciones de Recursos:** Escenarios donde la munición (T) es crítica o nula.
- **Saturación:** Pruebas de estrés con múltiples objetivos para verificar la efectividad de las validaciones en el tiempo de ejecución.

Con el objetivo de facilitar la auditoría técnica sin comprometer la legibilidad de este documento, se ha compilado un registro detallado que incluye los parámetros de entrada (N , M , T), la representación textual de los mapas de salida (sites.pgm) y los tiempos de ejecución cronometrados para cada uno de los nueve casos.

Puede acceder al registro completo de pruebas y resultados en el siguiente repositorio digital: <https://pastebin.com/0jNqDFEL>

Conclusiones

El desarrollo de este proyecto Vista de Cazador ha cumplido satisfactoriamente con todos los requerimientos operativos y tácticos planteados.

La arquitectura basada en punteros y clases encapsuladas permite al sistema adaptarse a cualquier dimensión de entrada, superando las limitaciones de las implementaciones estáticas tradicionales.

Se demostró que la fuerza bruta no es una opción viable en la guerra algorítmica. La implementación de estas validaciones lógicas transformó un problema de complejidad exponencial en una solución de tiempo polinómico para los casos promedio, garantizando la viabilidad del sistema en tiempo real.

El algoritmo respeta estrictamente la jerarquía de prioridades: maximización del daño (Área), economía de recursos (Menor Cantidad) y priorización de amenazas (Oscuridad), gracias al pre-ordenamiento estratégico de los datos.

En definitiva, el sistema no solo "ve" el campo de batalla, sino que "entiende" la mejor manera de resolverlo, tal como se exigió.